DA3408 Assignment 5

SwiftParcel ETA Service

Test machine. Ubuntu VM with 6 vCPUs, 11 GiB RAM, and Python 3.14.4. All runs used the CPU.

Part A. REST service

A1 and A2. API and tests

I ran the FastAPI service with model v1. A valid request returned a prediction of 50.1 minutes, and the response contained the same request ID as the request. The service returned HTTP 422 for an infeasible delivery and exposed Prometheus counters and a latency histogram. All 15 pytest cases passed, including a simulated inference failure.

A3. Load testing

The client and server ran on the same VM. For each worker count, I used a two-second warm-up, then ran the test for four seconds at each concurrency level. Both sweeps used batch size 1 with access logging disabled.

Workers	Clients	Requests	Requests/s	p50 (ms)	p95 (ms)	p99 (ms)	Errors
1	1	3653	913.2	1.1	1.3	1.8	0
1	2	4552	1138.0	1.7	1.9	3.2	0
1	4	4121	1030.2	3.6	6.2	7.8	0
1	8	3580	895.0	8.3	15.2	21.6	0
1	16	3522	880.5	17.2	30.1	35.7	0
2	1	3867	966.8	1.0	1.1	1.4	0
2	2	4677	1169.2	1.7	1.8	2.2	0
2	4	4171	1042.8	3.6	6.0	7.5	0
2	8	7199	1799.8	3.5	9.6	13.0	0
2	16	7514	1878.5	7.7	15.9	21.7	0

Saturation and capacity. With one worker, throughput peaked at two clients and fell at higher concurrency. Using Little's Law, I estimated mean response time as concurrency divided by throughput. It rose from 1.76 ms at two clients to 18.17 ms at 16 clients, consistent with saturation near two clients in this sweep. All measured settings met the 80 ms p95 target. The highest rate with one worker was 1,138.0 requests/s.

Batch size. I kept one worker and one client. The amortized time per row is 1000/(requests/s $\times$ batch size). With batch size 100, the deliveries share the HTTP overhead and go through one prediction call.

Batch size	Requests/s	Rows/s	Amortized ms/row
1	913.2	913.2	1.095
100	691.0	69,100	0.0145

Second worker. I repeated the sweep with two workers to let the server use another core. Peak measured throughput within the latency target rose to 1,878.5 requests/s, a 65.1% gain. At 16 clients, throughput was 2.13 times as high and p95 fell from 30.1 to 15.9 ms.

Why p95 and p99. The mean can hide slow responses when most requests finish quickly. The p95 and p99 show tail latency, and p95 is the metric used in the 80 ms target.

Part B. MLflow model promotion

B1. Model registry

I used MLflow 3.18.0 with a SQLite tracking store. Both pyfunc models were logged with an input example, an input and output signature, and pinned dependencies. I loaded each alias and checked the model label in its prediction.

Registered model	Registry version	Alias	Bundle
eta-model	1	prod	v1
eta-model	2	candidate	v2

B2. Promotion gate

All 14 gate tests passed. I evaluated both models on the same 2,000 fresh deliveries and measured in-process p95 on 200 single-row predictions per model in each run.

Metric	prod (v1)	candidate (v2)
Overall MAE (min)	11.955	6.105
Rush-hour MAE (min)	19.680	8.580
Rain MAE (min)	11.623	6.285
In-process p95, default run (ms)	0.926	0.914
In-process p95, stricter run (ms)	0.944	0.915

The default gate required at least 3% lower overall MAE, candidate p95 of at most 10 ms, and at most 5% MAE regression on each of the two slices. The candidate passed all four checks. Its overall MAE improved by 48.9%, so the gate rejected it when I raised the required improvement to 60%. Only the overall MAE check failed in that run.

B3. Shadow comparison

I sent the same 200 fresh deliveries to both MLflow servers, one row per HTTP request, and verified each response's model label.

Measurement	Result
Paired requests	200
Mean absolute difference between predictions	9.4845 min
Pairs with a difference greater than 10 min	70/200 (35.0%)
prod HTTP p95 on port 5101	2.896 ms
candidate HTTP p95 on port 5102	3.009 ms

I would promote v2 under the default policy because it improved overall and slice MAE, with HTTP p95 close to v1. During rollout I would track the 35% of deliveries for which the forecasts differed by more than 10 minutes. MLflow resolves the alias when loading the model, so an already running server keeps using its loaded model. One rollout option is to warm up replacement replicas, shift traffic to them after readiness checks, and drain requests on the old replicas. Another is to load and warm the new model in the background, then switch the model reference for new requests while in-flight requests finish on the old model. [MLflow registry workflow]

Part C. Ray Serve canary

C1. Routing and deployments

I used SHA-256 to assign each customer ID to a stable bucket and compared the bucket with the canary weight. The `X-Beta: 1` header overrides the assignment for that request. The demo customer received an ETA of 50.1 minutes from v1 on a normal request and 51.2 minutes from v2 with the header.

Deployment	Replicas	CPUs per replica
Router	1	0.25
Production v1	2	0.25
Candidate v2	1 to 3	0.25

C2. Verification

All five checks passed on a two-CPU Ray cluster.

Check	Observed result
Candidate share at 20% weight	192/1,000 customers (19.2%), within the 16-24% range.
Repeat requests	All 200 customers kept their original arm.
Identical redeployment	All 200 customers kept their original arm.
Weight increased to 50%	All 192 original v2 customers stayed on v2. In total, 488/1,000 customers went to v2.
Header override	All 50 v1 customers went to v2 with the header and returned to v1 on a regular request.

C3. Short answers

1. Hash function. Python salts string hashes by default, so `hash(customer_id)` can change between processes or restarts. SHA-256 gives the same customer ID the same bucket across replicas and redeployments. [Python data model]

2. Stickiness. A customer keeps seeing estimates from the same model across repeat requests. For the experiment, stable assignments let me compare outcomes by customer while keeping exposure to the two models separate.

3. Autoscaling. I set a target of two ongoing requests per candidate replica and allowed up to four to give the autoscaler room to react. The five-second upscale delay responds to sustained load, while the 30-second downscale delay avoids removing capacity during short dips. One candidate replica stays warm, and a maximum of three keeps the total reservation at 1.5 CPUs, including both production replicas and the router. [Ray Serve autoscaling]

Part D. Release smoke test

D1. Smoke test and canary abort criteria

All six checks passed on v1. Increasing distance raised the ETA from 50.1 to 127.4 minutes, and increasing traffic raised it from 50.1 to 73.6 minutes. A zero-distance request returned HTTP 422. The p95 for 30 single-row HTTP requests was 2.076 ms, within the 80 ms limit. The successful run returned exit code 0, and deliberately expecting v2 from the v1 server returned exit code 1.

Abort criteria for a 10% v2 canary

Metric	Threshold	Window
Server errors	More than 1% of valid v2 requests return HTTP 5xx or time out.	Rolling 5 min with at least 100 eligible v2 requests.
HTTP latency	Candidate p95 exceeds 80 ms.	Rolling 5 min with at least 100 successful v2 predictions.
Prediction sanity	Two consecutive forced-v2 probes fail the distance or traffic check.	Rolling 5 min with one probe per minute.
ETA accuracy	v2 MAE exceeds 105% of v1 shadow MAE on the same completed deliveries.	Rolling 24 h with at least 100 paired deliveries with observed times.

Not tested

Load-driven Ray scaling, live 10% v2 canary monitoring, and the proposed zero-downtime rollouts were not exercised. The live smoke test used v1.